INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DO CEARÁ
CAMPUS BOA VIAGEM
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS

Emyliano Carvalho Costa

Sistema Único de Transportes Escolares (SUTE)

Sistema web orientado a objetos em PHP

Relatório do trabalho individual da disciplina Programação Web I
Professor: Prof. Dr. Renato William Rodrigues de Souza

Boa Viagem, CE
2026

Sumário

1 Introdução **2**

2 Requisitos **2**

2.1 Funcionalidades 2

2.2 Regras de negócio 2

3 Modelagem **3**

3.1 Herança 3

3.2 Composição 3

4 Implementação **3**

4.1 Encapsulamento e construtores (OO1 e OO4) 4

4.2 Herança e sobrescrita (OO2) 4

4.3 Polimorfismo (OO3) 4

4.4 Regras de negócio com exceções (OO5) 5

4.5 Composição (OO6) 5

4.6 Organização em arquivos e conversão para JSON (OO7 e OO8) 5

5 Camada web **6**

6 Testes **6**

7 Ferramentas utilizadas **7**

8 Uso de LLM **8**

9 Dificuldades, aprendizados e conclusão **8**

1 Introdução

O SUTE (Sistema Único de Transportes Escolares) foi desenvolvido para auxiliar prefeituras na gestão do transporte público escolar municipal. O sistema resolve o problema do controle de passageiros diários e o cálculo do custo de repasse financeiro, servindo a gestores públicos e empresas de transporte terceirizadas.

Este relatório detalha o desenvolvimento do SUTE. A Seção 2 apresenta os requisitos e regras de negócio. A Seção 3 exibe a modelagem orientada a objetos. A Seção 4 detalha a implementação do código, seguida pela arquitetura da camada web na Seção 5. Por fim, as Seções 6 a 9 abordam os testes, ferramentas utilizadas, o uso de inteligência artificial e as conclusões do projeto.

2 Requisitos

2.1 Funcionalidades

O sistema possui uma interface web com menu de navegação que permite:

- Visualizar as rotas ativas, incluindo a lista de alunos cadastrados, o veículo vinculado e o custo mensal total para a prefeitura.
- Gerenciar uma frota de veículos (Vans e Ônibus) e associá-los de forma dinâmica às rotas.
- Cadastrar, mover e excluir alunos nas rotas através de formulários POST com validação estrita de dados.
- Exportar os dados completos do sistema em formato JSON por meio de uma rota específica.
- Armazenar os cadastros de forma persistente em um arquivo local (`dados.json`), garantindo que os dados não sejam perdidos entre requisições.

2.2 Regras de negócio

Nº	Regra	Onde está (classe e método)
1	O nome do aluno deve ter entre 3 e 80 letras, sem números	`Aluno::setNome()`
2	A matrícula deve conter apenas números (até 20 dígitos)	`Aluno::setMatricula()`
3	A placa do veículo deve seguir o formato do DETRAN	`Veiculo::__construct()`
4	A capacidade do veículo deve ser entre 1 e 100 lugares	`Veiculo::__construct()`
5	A distância da rota deve ser maior que zero	`Rota::__construct()`

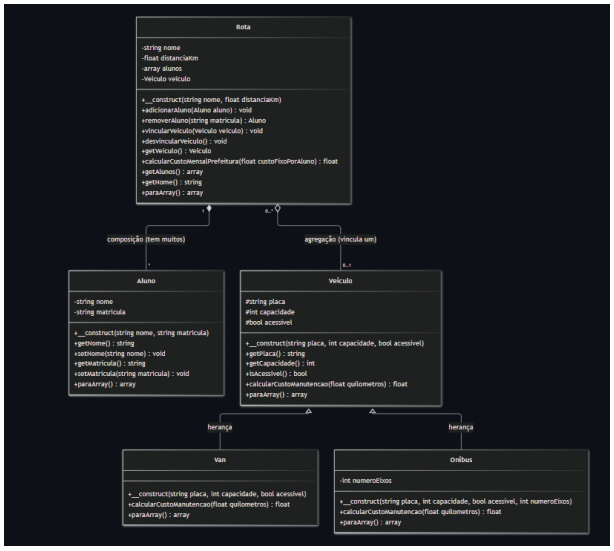


Figura 1: Enter Caption

3 Modelagem

3.1 Herança

A classe mãe `Veiculo` agrupa atributos e comportamentos comuns a qualquer transporte (placa, capacidade, acessibilidade). As classes filhas `Van` e `Onibus` herdam essa estrutura pois a relação "é um" faz sentido (Uma Van *é um* Veículo). A herança justifica-se pela sobrescrita do método `calcularCustoManutencao`, onde cada filha aplica a sua própria fórmula de custos operacionais (polimorfismo).

3.2 Composição

A classe `Rota` guarda um *array* de objetos da classe `Aluno`. A relação é do tipo "tem muitos", pois uma Rota *tem muitos* alunos alocados a ela. Essa composição é vital para o sistema, pois o método `calcularCustoMensalPrefeitura()` precisa iterar sobre esta lista de alunos para determinar o valor de repasse financeiro do trajeto. A rota também possui uma agregação simples ("tem um") com a classe `Veiculo`.

4 Implementação

Nesta seção, apresentamos trechos de código que comprovam o cumprimento dos requisitos de Orientação a Objetos (OO1 a OO8). O código completo encontra-se no repositório.

4.1 Encapsulamento e construtores (OO1 e OO4)

A classe `Aluno` demonstra o uso de tipagem estrita, atributos privados e um construtor que utiliza *setters* para garantir a validação na criação do objeto:

```
declare(strict_types=1);

class Aluno {
    private string $nome;
    private string $matricula;

    public function __construct(string $nome, string $matricula) {
        $this->setNome($nome); // Validação de tamanho e letras
        $this->setMatricula($matricula); // Validação numérica
    }
}
```

4.2 Herança e sobrescrita (OO2)

A classe `Onibus` estende a classe mãe `Veiculo`, chama o construtor pai utilizando `parent::__construu` e sobrescreve o cálculo de manutenção:

```
class Onibus extends Veiculo {
    private int $numeroEixos;

    public function __construct(string $placa, int $capacidade, bool $acessivel, int
    $numeroEixos) {
        parent::__construct($placa, $capacidade, $acessivel);
        $this->numeroEixos = $numeroEixos;
    }

    public function calcularCustoManutencao(float $quilometros): float {
        $custoBase = parent::calcularCustoManutencao($quilometros);
        return $custoBase * 1.5 + ($this->numeroEixos * 120.00);
    }
}
```

4.3 Polimorfismo (OO3)

No sistema, um laço `foreach` percorre diferentes tipos de veículos iterando sobre um único array de frota. O método é o mesmo, mas a resposta é específica de cada classe:

```
$frota = [
    new Van("ABC-1234", 15, false),
    new Onibus("XYZ-9876", 45, true, 2)
];
```

```php
foreach ($frota as $veiculo) {
    // Polimorfismo: Van e nibus calculam de forma diferente
    $custo = $veiculo->calcularCustoManutencao(100.0);
}
```

4.4 Regras de negócio com exceções (OO5)

Na classe `Veiculo`, regras como validação de placa lançam uma `Exception`. No `index.php`, essa exceção é capturada (`catch`) para exibir a mensagem:

```php
// src/Veiculo.php
if (!preg_match('/^[A-Z]{3}[0-9][A-Z0-9][0-9]{2}$/', $placaFormatada)) {
    throw new Exception("Erro: A placa deve seguir o formato DETRAN.");
}

// public/index.php
try {
    $frota[] = new Van($placa, $capacidade, $acessivel);
} catch (Exception $e) {
    $erro = true;
    $mensagem = $e->getMessage();
}
```

4.5 Composição (OO6)

A classe `Rota` possui métodos para adicionar e remover objetos da lista de alunos, e calcular os custos com base na quantidade:

```php
class Rota {
    private array $alunos = [];

    public function adicionarAluno(Aluno $aluno): void {
        $this->alunos[] = $aluno;
    }

    public function calcularCustoMensalPrefeitura(float $custoFixoPorAluno): float {
        return count($this->alunos) * $custoFixoPorAluno;
    }
}
```

4.6 Organização em arquivos e conversão para JSON (OO7 e OO8)

As classes estão separadas na pasta `src/`. Todas implementam o método `paraArray()` estruturando os dados para a exportação JSON:

```
public function paraArray(): array {
    $alunosArray = array_map(fn($a) => $a->paraArray(), $this->alunos);

    return [
        'nome' => $this->nome,
        'distanciaKm' => $this->distanciaKm,
        'totalAlunos' => count($this->alunos),
        'alunos' => $alunosArray,
        'veiculo' => $this->veiculo ? $this->veiculo->paraArray() : null
    ];
}
```

5 Camada web

A aplicação utiliza o padrão *Front Controller* no arquivo `index.php`. As rotas de página são controladas via *query string* (`?pagina=home`, `cadastro`, `veiculos`, `rotas`). Se o usuário solicitar uma página inválida, o sistema atribui '404' à variável e envia o cabeçalho via `http_response_code(404)`.

As interações são processadas via POST, capturando campos como nome, matrícula, tipo e placa. Se as regras de negócio dispararem uma exceção no momento de instanciar os objetos, a falha é apanhada num `try/catch` e exibida numa div de segurança na tela. Os dados exibidos usam `htmlspecialchars()` para prevenir XSS.

Abaixo, a saída real da exportação na rota `?formato=json` via `curl`:

```
> curl.exe -i http://localhost:8000/?formato=json
HTTP/1.1 200 OK
Host: localhost:8000
Connection: close
X-Powered-By: PHP/8.3.0
Content-Type: application/json; charset=utf-8

{"rotas":[{"nome":"Zona Rural - Centro","distanciaKm":12.5,"totalAlunos":0,"alunos
    ":[],"veiculo":null}],"frota":[{"placa":"ABC1234","capacidade":15,"acessivel":
    false,"tipo":"Van"}]}
```

6 Testes

O arquivo `teste.php` invoca diretamente as classes no terminal para comprovar a robustez do encapsulamento contra dados corrompidos. Abaixo a saída real:

```
> php teste.php
--- TESTES DO SISTEMA SUTE ---
```

```
Teste 1: Nome invalido (Regra OO5)
OK, recusado: Erro: O nome do aluno não pode conter números.

Teste 2: Matricula com letras (Regra OO5)
OK, recusado: Erro: A matrícula deve conter apenas números.

Teste 3: Placa fora do padrao (Regra OO5)
OK, recusado: Erro: A placa deve seguir o formato DETRAN (Ex: ABC-1234 ou ABC1D23).

Teste 4: Polimorfismo (Regra OO3)
Veiculo ABC1234 (Van): R$ 160,00
Veiculo XYZ9876 (Onibus): R$ 580,00

Teste 5: Composicao (Regra OO6)
Total de alunos na rota 'Zona Rural - Centro': 2
Custo mensal para a prefeitura: R$ 170,00
```

O que cada linha comprova:

- **Testes 1 a 3:** Validam o bloqueio eficaz nos *setters* (OO5), recusando formatações inválidas de texto, tamanho numérico e expressões regulares das placas Detran.
- **Teste 4:** O polimorfismo (OO3) foi comprovado ao iterar o array de veículos. O método `calcularCustoManutencao` obteve o retorno de R$160 para a Van e R$580 para o Ônibus, processando internamente a regra de acessibilidade e eixos.
- **Teste 5:** A composição (OO6) funcionou ao instanciar os objetos alunos e vinculá-los à lista privada da rota para realizar um cálculo aritmético da sua lotação.

7 Ferramentas utilizadas

Ferramenta	**Para que foi usada**
PHP 8.3	Desenvolvimento da lógica *backend*, *scripts* e roteamento interno
VS Code	Edição do código-fonte e gestão de diretórios
Git e GitHub	Versionamento do código (*commits*, *branches*) e entrega do projeto
curl	Teste de chamadas HTTP e depuração do cabeçalho da rota JSON
Navegador e DevTools	Testes de *frontend*, interface web, responsividade do CSS e formulários
LLM: Gemini	Assistente de pareamento de código, auxílio no CSS responsivo e LaTeX

8 Uso de LLM

O diário detalhado encontra-se no arquivo `docs/llm.md`. Utilizei o Gemini como assistente técnico focado em refatoração e otimização. A IA foi crucial para desenvolver a responsividade do layout CSS e me auxiliar na persistência dos dados usando `json_decode` / `file_put_contents`.

Durante o desenvolvimento das regras de negócio, a IA inicialmente sugeriu um cálculo cobrando "mensalidade"individual dos alunos. Eu intervi e corrigi a instrução, informando que como o transporte era municipal (Prefeitura), o cálculo deveria focar no repasse de custo público por lotação, adequando assim o sistema à realidade do domínio de negócio.

A LLM foi fundamental no ganho de produtividade, especialmente ao ensinar sobre tipagens estritas no PHP e uso de expressões regulares (`preg_match`) para o modelo de placas do DETRAN. No entanto, ela poderia ter atrapalhado no momento em que alteramos os construtores da hierarquia para incluir o parâmetro de "acessibilidade"; copiar e colar os trechos sem apagar o arquivo de banco de dados JSON antigo resultou em erros graves (*TypeError*) que exigiram depuração manual.

9 Dificuldades, aprendizados e conclusão

A principal dificuldade enfrentada durante a implementação do projeto foi realizar o vínculo cruzado na persistência de dados. Fazer com que o sistema web salvasse o veículo e os alunos de forma serializada no arquivo `dados.json` e, no momento de recarregar a página, reconstruísse os objetos hidratando-os novamente na classe mãe `Rota` exigiu um manuseio cuidadoso de manipulação de *arrays* associativos no PHP.

O aprendizado mais enriquecedor foi a aplicação prática do polimorfismo integrado ao *frontend*, ao invés de apenas testá-lo no terminal. Ver o sistema imprimir dinamicamente cálculos de veículos totalmente diferentes utilizando a mesma chamada de função na interface foi gratificante. Se começasse de novo, adotaria a estrutura de uma base de dados relacional leve (como SQLite) em vez do arquivo JSON para simplificar a remoção e o vínculo de instâncias via chave estrangeira.

Links

- Repositório: https://github.com/Emyyliano/pweb1-trabalho-oo